Protocol memo - Practice 2

Team: Team 1 | **Date:** 25 October 2026

P2 roles: R1 Diana Luque; R2 Francisco Moya; R3 Claudia Martínez; R4 Fernando Caceres; R5 Marina García.

1. Protocol construction

Cyber uses five-fold `StratifiedGroupKFold` on `feature_group`, stratified by `attack_code` as far as grouping permits. Storage uses five-fold `GroupKFold` on `config_group`. Both own constructions use `shuffle=True, random_state=2627`; the row-wise comparators also shuffle with 2627. The project seed is printed first and is not searched or selected. KNN and the scalers themselves consume no random state.

Property	Cyber	Storage
Development rows / groups	34,928 / 34,883	12,190 / 2,438
Canonical validation rows, folds 0-4	6,986; 6,986; 6,985; 6,986; 6,985	2,440; 2,440; 2,440; 2,435; 2,435
Generalisation unit	Unseen exact feature group	Unseen configuration; all five replicates stay together

2. Verified invariants

The constructed folds print zero shared training/validation groups in every fold. Canonical checks also print zero groups assigned to multiple folds; all rows are validated exactly once. All ten cyber classes occur in every validation fold. Ransomware has 208 observations, with canonical supports 42, 42, 41, 41 and 42. Storage has exactly five replicates per configuration. A nonzero shared-group count, incomplete row coverage or a missing validation class would be a failed check, not an acceptable result to ignore.

3. What the split comparison measured

The P2.2 comparison uses the course-fixed KNN with five neighbours. Cyber preprocessing is one-hot for three nominal predictors, then log1p and standardisation for eight numeric predictors. Storage uses one-hot for four nominal predictors and standardisation for five numeric predictors. Learned preprocessing is fitted inside each training fold. Only the partition protocol changes; the grouped results below use our own construction before canonical adoption.

Challenge / metric	Grouped mean	Row-wise mean	Grouped minus row-wise
Cyber / macro F1	0.9442	0.9443	-0.0001
Storage / MAE(z)	0.2124	0.0809	+0.1315

Storage's row-wise error is optimistic: training can contain zero-distance replicates of a validation configuration. The grouped error is 2.62 times larger. Cyber's measured gap is small; its sign is not prescribed. Splitting identical feature groups still changes what generalisation is measured, irrespective of the score gap. Fold labels under different protocols do not identify the same validation rows. Fold dispersion is descriptive, not a confidence interval, significance test or equivalence test.

4. Target representation

From P2.3 through P5, Challenge B models use `z = log10(latency_ns)`. This decision concerns scale, loss and measurement structure, not merely a better score: the five-neighbour KNN gives MAE(z) about 0.2103 when fitted on raw latency and 0.2091 when fitted on z on the same canonical folds.

Latency spans about 4.3 orders of magnitude, from 1.16 x 10^5 to 2.53 x 10^9 ns. The raw mean is about 23 times the median and the raw skewness is about 9; after log10, skewness is about 1. Raw arithmetic averages are strongly influenced by the slowest observations. The log reduces this imbalance without requiring a normal target distribution.

MAE in nanoseconds measures absolute cost; it gives the same numerical weight to a fixed ns error at every latency level. MAE(z) instead measures the magnitude of a log-ratio: `abs(z - z_hat) = abs(log10(predicted_ns / observed_ns))`. The same multiplicative error therefore has the same cost for fast and slow configurations. In the fixed KNN diagnostic, the slowest fifth accounts for about 92% of raw-nanosecond error; that motivates retaining a physical metric alongside the primary log-scale metric.

Replicate noise also changes with latency: from the fastest to slowest configuration quintile, its typical standard deviation grows about 1,300 times in ns but about 9 times in z. This is consistent with an important multiplicative component; the log reduces, but does not remove, dependence on the latency level. Finally, `10 ** z_hat` is mathematically positive. Floating-point outputs must still be checked for finiteness and strict positivity; a raw-target model need not preserve positivity when extrapolating.

5. Metric contract

Challenge	Primary and supporting metrics	Engineering interpretation
Cyber	Primary macro F1; weighted F1, accuracy; per-class precision, recall and F1 with support; semantic confusion matrix	Equal class weight prevents the rare class from disappearing in an overall average. Secondary concern: ransomware misses versus analyst false-alarm workload.
Storage	Primary MAE(z); RMSE(z), R2(z); one MAE in ns	Log error compares multiplicative deviations; MAE_ns retains absolute physical cost and highlights errors on slow configurations.

Log metrics use z and z_hat directly. Physical metrics first invert each prediction: `predicted_ns = 10 ** z_hat`; then MAE_ns compares it with original `latency_ns`. `10 ** MAE(z)` is a multiplicative factor, never an error in ns or a guaranteed prediction-coverage bound. The P2.5 ransomware exercise uses out-of-fold probabilities and the five prescribed thresholds, plus ten-class argmax as a separate reference; it does not replace macro-F1 model selection.

6. Frozen predictive-information basis

The primary route retains all eleven distributed cyber predictors and all nine storage predictors. Cyber declares log1p on its eight numeric predictors; the P2 pipelines use one-hot nominal encoding and standardisation. Storage freezes the target z after P2.3. The typed register is `plantillas/features.md`. `row_id`, `feature_group`, `config_group` and `fold_id` never enter an estimator; excluded `iops` is not reintroduced. P2 introduces no derived predictive feature or extension row.

Later families may use justified representations of this same source information. Primary-route evidence is a comparison of pipelines unless the predictor representation is held constant or earlier families are rerun on the same representation. The folds, metric and source information remain fixed. Scores selected on these same development folds are not independent external-test estimates.

7. Canonical adoption and reproducibility

The own constructions and their checks are retained before canonical files are read: cell IDs `cyber-028`/`cyber-030` and `storage-025`/`storage-027`. Canonical comparison occurs at `cyber-038` and `storage-033`; adoption at `cyber-041` and `storage-036`. Cyber reproduces the canonical partition exactly. Storage differs in configuration membership, even after relabelling, while preserving grouping and fold sizes. No alternative seed was searched. From adoption onward, all scored model results use the course sidecars, not the own construction.

Both integrated notebooks ran from beginning to end in fresh Jupyter kernels with saved outputs: 34 cyber and 45 storage code cells, with no error outputs. Both development-only dry runs returned `valid`; storage also checks physical magnitude, finiteness and positivity. No final-test features were read. Verification used Python 3.12.14, NumPy 2.5.4, pandas 3.0.1, scikit-learn 1.9.1 and matplotlib 3.11.2. These versions are recorded; the original setup note mentioning Python 3.14 is not an assertion that this run used it. This protocol memo is retained unchanged for the defence pack.

Data attribution and references

Cyber is the course-derived UNSW TON_IoT Network dataset; the course removed target/identity shortcuts and retained group-aware development data. Storage derives from HSE-LAMBDA's Data Storage System Performance dataset (Apache-2.0): *Performance Modeling of Data Storage Systems Using Generative Models*, IEEE Access 13, 49643-49658 (2025), doi:10.1109/ACCESS.2025.3552409. Full derivations and terms are in the two `PROVENANCE.md` records. The eight required TON_IoT references are:

1. N. Moustafa. "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets." Sustainable Cities and Society (2021), 102994.

2. T. M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa and F. T. H. den Hartog. "ToN IoT - The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion datasets." IEEE Internet of Things Journal (2021).

3. A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood and A. Anwar. "TON_IoT telemetry dataset: a new generation dataset of IoT and IIoT for data-driven Intrusion Detection Systems." IEEE Access 8 (2020), 165130-165150.

4. N. Moustafa, M. Keshk, E. Debie and H. Janicke. "Federated TON_IoT Windows Datasets for Evaluating AI-Based Security Applications." TrustCom (2020), 848-855. doi:10.1109/TrustCom50675.2020.00114.

5. N. Moustafa, M. Ahmed and S. Ahmed. "Data Analytics-Enabled Intrusion Detection: Evaluations of ToN_IoT Linux Datasets." TrustCom (2020), 727-735. doi:10.1109/TrustCom50675.2020.00100.

6. N. Moustafa. "New Generations of Internet of Things Datasets for Cybersecurity Applications based Machine Learning: TON_IoT Datasets." eResearch Australasia, Brisbane (2019).

7. N. Moustafa. "A systemic IoT-Fog-Cloud architecture for big-data analytics and cyber security systems: a review of fog computing." arXiv:1906.01055 (2019).

8. J. Ashraf, M. Keshk, N. Moustafa, M. Abdel-Basset, H. Khurshid, A. D. Bakhshi and R. R. Mostafa. "IoTBoT-IDS: A Novel Statistical Learning-enabled Botnet Detection Framework for Protecting Networks of Smart Cities." Sustainable Cities and Society (2021), 103041.